

UNITAT 7 · SETMANES 10–12

Fils i concurrència

📅 Setmanes 10–12 · dates per confirmar </> Laboratori TD6 (resolt + repte IA)

📺 5 animacions

LABORATORI TD6 · RESOLT

Fils i concurrència

Diversos fils comparteixen la memòria d'un procés: és ràpid i còmode, i també és l'origen dels errors més difícils de la programació. Aquest laboratori els provoca a propòsit i els corregeix amb mutex i semàfors.

EXERCICIS

- | | | | |
|---|--|---|--|
| 1 | Crear i esperar fils | 4 | El sopar dels filòsofs |
| 2 | Venedors d'entrades: la cursa i dues solucions | 5 | Exploració: cost d'un fil, Peterson en un x86 i el futex (nou) |
| 3 | Productors i consumidors | | |

TD6 · 1

● Resolt · provat

Crear i esperar fils

Enunciat. Crea 8 fils amb `pthread_create` que executin `start` i imprimeixin el pid del procés (passat com a argument). El fil principal els espera tots amb `pthread_join`. Afegeix una variable global i una de local a `start`, incrementa-les i imprimeix-ne les adreces i els valors.

Solució



C tds/td6/threads.c

```

/*
 * TD6 - exercise 1: create 8 threads, wait for them, and compare a global
 and a local variable.
 * All threads share the global `shared` (same address) but each has its own
 `local` (on its
 * own stack): the addresses printed show it. `shared++` is a race: see
 exercise 2.
 * Build: gcc -pthread (the Makefile already does it).
 */
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define NTHREADS 8

static int shared = 0;

static void *start(void *arg)
{
    pid_t pid = (pid_t)(intptr_t)arg;      /* the pid travels inside the
void * */
    int local = 0;

    shared++;
    local++;
    printf("thread %lu of process %d: shared[%p] = %d, local[%p] = %d\n",
        (unsigned long)pthread_self() % 100000, pid, (void *)&shared,
        shared, (void *)&local, local);
    return NULL;                          /* same as pthread_exit(NULL) */
}

int main(void)
{
    pthread_t tids[NTHREADS];
    pid_t pid = getpid();

    for (int i = 0; i < NTHREADS; i++) {
        int rc = pthread_create(&tids[i], NULL, start, (void *)
(intptr_t)pid);

```

```

        exit_if(rc != 0, "pthread_create"); /* pthread functions return the
error code */
    }
    for (int i = 0; i < NTHREADS; i++)
        exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");

    printf("process %d: all %d threads finished, shared = %d\n", pid,
NTHREADS, shared);
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1,
local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4,
local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5,
local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2,
local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3,
local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6,
local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7,
local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8,
local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8

```

Tots els fils imprimeixen el **mateix pid** (són un sol procés) i la **mateixa adreça** per a `shared`, però cada `local` és en una adreça diferent: cada fil té la seva pila. El valor de `shared` surt desordenat i fins i tot es podria repetir: `shared++` no és atòmic.

Venedors d'entrades: la cursa i dues solucions

Enunciat. Per a una festa de la facultat, 8 estudiants (fils) venen en paral·lel les entrades d'un comptador global: mentre en queden, criden `sched_yield`, en resten una i compten la seva venda; al final retornen quantes n'han venut amb `pthread_exit` i el fil principal les suma. Prova quantitats creixents. Què passa? On és la secció crítica? Protegeix-la amb un semàfor i amb un mutex.

Solució

C tds/td6/tickets_race.c

```
static volatile int tickets = 1000;    /* volatile only prevents caching in a
register:

                                     it does NOT make the accesses
atomic */

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    while (tickets > 0) {               /* critical section starts here ...
*/
        sold++;
        sched_yield();                 /* do not touch this line
*/
        tickets--;                     /* ... and ends here
*/
    }
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;    /* same as pthread_exit((void *)sold)
*/
}
```

Sortida real

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

Es venen més entrades de les que hi ha i el comptador acaba negatiu. La secció crítica és tot el tram des de `tickets > 0` fins a `tickets--` : entre la comprovació i la resta un altre fil pot vendre l'última entrada, i `tickets--` és en si mateix llegir-restar-escriure, tres passos que es poden intercalar.

C tds/td6/tickets_mutex.c

```
static int tickets = 10000;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    for (;;) {
        pthread_mutex_lock(&lock);
        if (tickets ≤ 0) {                                /* checked with the lock held */
            pthread_mutex_unlock(&lock);
            break;
        }
        sold++;
        sched_yield();
        tickets--;
        pthread_mutex_unlock(&lock);
    }
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;
}
```

Sortida real

```
$ ./tickets_mutex 100000 | tail -3
[3] I sold 11577 tickets
[2] I sold 11338 tickets
100000 tickets sold out of 100000 (tickets = 0)
```

C tds/td6/tickets_sem.c

```
static sem_t available;

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    while (sem_trywait(&available) == 0) {    /* took one ticket */
        sold++;
        sched_yield();
    }
    exit_if(errno != EAGAIN, "sem_trywait");    /* EAGAIN = no tickets left,
    anything else is a bug */
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;
}
```

Sortida real

```
$ ./tickets_sem 100000 | tail -3
[6] I sold 14469 tickets
[2] I sold 11537 tickets
100000 tickets sold out of 100000 (tickets = 0)
```

Amb el **mutex**, la comprovació i la resta es fan amb el pany agafat (i la comprovació es repeteix a dins: el valor llegit abans d'agafar-lo pot estar desfasat). Amb el **semàfor** el disseny és encara millor: el semàfor és el comptador d'entrades, i `sem_trywait` «comprova i agafa» en una sola operació indivisible.

Corregit

La solució heretada amb mutex no compilava (`nbtickets_venduss`) i llegia el comptador fora del pany.

Animació A7.1 · Condició de cursa →

Productors i consumidors

Enunciat. (a) Un productor suma 1 a cada cel·la d'un vector **A** i un consumidor suma totes les cel·les, **LOOP** vegades cadascun. Fes-ne dos fils que **s'alternin**, cadascun amb accés exclusiu a tot el vector. (b) Ara hi ha 1 productor i 4 consumidors: cada valor produït s'ha de consumir exactament una vegada. Quines seccions crítiques noves apareixen?

Solució

C tds/td6/prodcons.c

```
/*
 * TD6 - exercise 3 (a): one producer and one consumer take turns on the
 * WHOLE array.
 * The producer adds 1 to every cell, then the consumer sums every cell; LOOP
 * rounds.
 * Expected sum: SIZE * (1 + 2 + ... + LOOP) = SIZE * LOOP * (LOOP + 1) / 2.
 *
 * Two semaphores implement the turn-taking ("baton passing"):
 * can_produce = 1 at start (the producer goes first), can_consume = 0.
 * The critical section is the traversal of the array: while one thread walks
 * it, the
 * other must not touch it. With the two semaphores, at most one of them is
 * inside.
 */
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define SIZE 10000
#define LOOP 10

static int A[SIZE];
static long sum = 0;
static sem_t can_produce, can_consume;

/* "Opaque" functions of the statement: they must not be modified. */
static int consume(int *value) { usleep(1); return *value; }
```

```

static void produce(int v, int *value) { usleep(1); *value += v; }

static void *producer(void *arg)
{
    (void)arg;
    for (int i = 0; i < LOOP; i++) {
        sem_wait(&can_produce);                                /* my turn */
        for (int j = 0; j < SIZE; j++)
            produce(1, &A[j]);
        printf("producer: round %d written\n", i + 1);
        sem_post(&can_consume);                                /* your turn */
    }
    return NULL;
}

static void *consumer(void *arg)
{
    (void)arg;
    for (int i = 0; i < LOOP; i++) {
        sem_wait(&can_consume);
        for (int j = 0; j < SIZE; j++)
            sum += consume(&A[j]);
        printf("consumer: sum after round %d = %ld\n", i + 1, sum);
        sem_post(&can_produce);
    }
    return NULL;
}

int main(void)
{
    pthread_t p, c;

    exit_if(sem_init(&can_produce, 0, 1) == -1 || sem_init(&can_consume, 0,
0) == -1, "sem_init");
    exit_if(pthread_create(&p, NULL, producer, NULL) != 0, "pthread_create");
    exit_if(pthread_create(&c, NULL, consumer, NULL) != 0, "pthread_create");
    exit_if(pthread_join(p, NULL) != 0 || pthread_join(c, NULL) != 0,
"pthread_join");

    long expected = (long)SIZE * LOOP * (LOOP + 1) / 2;
    printf("sum = %ld, expected %ld: %s\n", sum, expected, sum == expected ?
"OK" : "WRONG");
    sem_destroy(&can_produce);
    sem_destroy(&can_consume);

```

```
    return sum == expected ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Sortida real

```
$ ./prodcons | tail -4
consumer: sum after round 9 = 450000
producer: round 10 written
consumer: sum after round 10 = 550000
sum = 550000, expected 550000: OK
```

(a) Dos semàfors es passen el torn com un testimoni de relleus: `can_produce` comença a 1 i `can_consume` a 0. Cada fil espera el seu semàfor, recorre el vector i habilita l'altre. La suma esperada és $\text{SIZE} \times (1 + 2 + \dots + \text{LOOP}) = 10.000 \times 55$.

C tds/td6/prodcons_multi.c

```
/*
 * TD6 - exercise 3 (b): one producer, N consumers, and a bounded buffer (the
 * classic of the
 * lecture). The producer puts ITEMS values in a circular buffer of BUF
 * cells; the consumers
 * take them out and add them to a shared sum. Every value must be consumed
 * exactly once.
 *
 * empty = number of free cells (starts at BUF) - the producer waits on
 * it
 * full = number of used cells (starts at 0) - the consumers wait on
 * it
 * mutex = protects the `out` index shared by the consumers (new critical
 * section!)
 * sum_lock protects the shared sum
 * To stop, the producer sends one "poison pill" (value -1) per consumer.
 * ./prodcons_multi [consumers] [items] (default 4 consumers, 100000
 * items)
 */
#include <pthread.h>
#include <semaphore.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

#define BUF 16
```

```

#define POISON (-1)

static int buffer[BUF];
static int in = 0, out = 0;           /* in: producer only; out: shared
by consumers */
static sem_t empty, full;
static pthread_mutex_t out_lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t sum_lock = PTHREAD_MUTEX_INITIALIZER;
static long long sum = 0;
static int nconsumers = 4, items = 100000;

static void *producer(void *arg)
{
    (void)arg;
    for (int v = 1; v ≤ items + nconsumers; v++) {
        int value = v ≤ items ? v : POISON;
        sem_wait(&empty);           /* wait for a free cell */
        buffer[in] = value;         /* only one producer: no lock
needed for `in` */
        in = (in + 1) % BUF;
        sem_post(&full);           /* one more item available */
    }
    return NULL;
}

static void *consumer(void *arg)
{
    long mine = 0, count = 0;
    for (;;) {
        sem_wait(&full);
        pthread_mutex_lock(&out_lock); /* two consumers must not take
the same cell */
        int value = buffer[out];
        out = (out + 1) % BUF;
        pthread_mutex_unlock(&out_lock);
        sem_post(&empty);
        if (value == POISON)
            break;
        mine += value;
        count++;
    }
    pthread_mutex_lock(&sum_lock); /* add the partial sum once, not
per item */
    sum += mine;
    pthread_mutex_unlock(&sum_lock);
}

```

```

    printf("consumer %d: %ld items\n", (int)(intptr_t)arg, count);
    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc > 1) nconsumers = atoi(argv[1]);
    if (argc > 2) items = atoi(argv[2]);
    pthread_t prod, cons[nconsumers];

    exit_if(sem_init(&empty, 0, BUF) == -1 || sem_init(&full, 0, 0) == -1,
"sem_init");
    exit_if(pthread_create(&prod, NULL, producer, NULL) != 0,
"pthread_create");
    for (int i = 0; i < nconsumers; i++)
        exit_if(pthread_create(&cons[i], NULL, consumer, (void *) (intptr_t)i)
!= 0, "pthread_create");
    exit_if(pthread_join(prod, NULL) != 0, "pthread_join");
    for (int i = 0; i < nconsumers; i++)
        exit_if(pthread_join(cons[i], NULL) != 0, "pthread_join");

    long long expected = (long long)items * (items + 1) / 2;
    printf("sum = %lld, expected %lld: %s\n", sum, expected, sum == expected
? "OK" : "WRONG");
    return sum == expected ? EXIT_SUCCESS : EXIT_FAILURE;
}

```

Sortida real

```

$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK

```

(b) El problema clàssic de la memòria intermèdia limitada: `empty` compta les cel·les lliures, `full` les ocupades. Amb diversos consumidors apareix una secció crítica nova: l'índex `out`, protegit amb un mutex (si no, dos consumidors agafarien la mateixa cel·la). La suma de cada consumidor s'afegeix una vegada, sota un altre mutex. Per acabar, el productor envia una «píndola enverinada» (-1) per consumidor.



Coherència corregida

L'enunciat parlava de `producteur / consommateur` i les solucions heretades de `lecteur / ecrivain`; una no complia «accés exclusiu a tot el vector» i la de diversos consumidors feia espera activa amb `sem_trywait + usleep`. Ara l'enunciat, el codi i els noms coincideixen.

TD6 · 4

● Result · provat (nou)

El sopar dels filòsofs

Enunciat. 8 filòsofs seuen a una taula rodona amb un bastonet entre cada parella; per menjar, cadascun necessita el bastonet de la dreta i el de l'esquerra (un mutex per bastonet). Tots agafen primer el de la dreta, després el de l'esquerra, i repeteixen pensar–agafar–menjar–deixar. Què passa? Proposa un canvi que ho eviti.

Solució

C tds/td6/philosophers.c

```
/*
 * TD6 - exercise 4: the dining philosophers. N philosophers, N chopsticks
 (one mutex each).
 * Every philosopher takes the RIGHT chopstick, then the LEFT one.
 * DEMO_DEADLOCK (Makefile target philosophers_deadlock): everybody may
 hold the right
 * chopstick and wait forever for the left one: circular wait = deadlock. A
 pause between
 * the two lock operations makes it happen almost every time.
 * default (solution): break the circular wait by ordering the resources -
 every
 * philosopher takes the chopstick with the SMALLER number first. The last
 philosopher
 * (whose right chopstick is N-1 and left is 0) therefore starts with the
 left one.
 * ./philosophers [meals] (default 5 meals each)
 */
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
```

```

#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define N 8

static pthread_mutex_t chopstick[N];
static int meals = 5;

static void think(int id)
{
    printf("%2d: thinking\n", id);
    usleep((useconds_t)(random() % 2000));
    printf("%2d: hungry\n", id);
}

static void eat(int id)
{
    printf("%2d: eating\n", id);
    usleep((useconds_t)(random() % 2000));
    printf("%2d: full\n", id);
}

static void *philosopher(void *arg)
{
    int id = (int)(intptr_t)arg;
    int right = id, left = (id + 1) % N;
#ifdef DEMO_DEADLOCK
    int first = right, second = left;
#else
    int first = right < left ? right : left;      /* global order on the
    chopsticks */
    int second = right < left ? left : right;
#endif
    for (int m = 0; m < meals; m++) {
        think(id);
        pthread_mutex_lock(&chopstick[first]);
#ifdef DEMO_DEADLOCK
        usleep(20000);                          /* everybody grabs the
        first one... */
#endif
        pthread_mutex_lock(&chopstick[second]);
        eat(id);
        pthread_mutex_unlock(&chopstick[second]);
    }
}
  
```

```

        pthread_mutex_unlock(&chopstick[first]);
    }
    printf("%2d: done after %d meals\n", id, meals);
    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t tids[N];

    if (argc > 1)
        meals = atoi(argv[1]);
    setvbuf(stdout, NULL, _IOLBF, 0);    /* line by line, even into a pipe:
see who is stuck */
    srand(2026);
    for (int i = 0; i < N; i++)
        exit_if(pthread_mutex_init(&chopstick[i], NULL) != 0,
"pthread_mutex_init");
    for (int i = 0; i < N; i++)
        exit_if(pthread_create(&tids[i], NULL, philosopher, (void *)
(intptr_t)i) != 0, "pthread_create");
    for (int i = 0; i < N; i++)
        exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
    printf("dinner finished: %d philosophers x %d meals\n", N, meals);
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status:
${PIPESTATUS[0]} (124 = still blocked after 3 s)"
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)

```

Sortida real

```
$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals
```

Si tothom agafa primer el de la dreta, n'hi ha prou que els 8 ho facin alhora perquè cadascun esperi el bastonet que té el veí: **interbloqueig**. La sortida real mostra els 8 «afamats» i el programa encallat fins que **timeout** el mata. Es compleixen les quatre condicions de Coffman (exclusió mútua, retenció i espera, no expropiació i espera circular). La solució trenca l'**espera circular**: els bastonets s'ordenen i cada filòsof agafa primer el de número més baix; per tant, l'últim filòsof agafa primer el de l'esquerra i el cicle no es pot tancar.

[Animació A7.3 · Interbloqueig →](#)

TD6 · 5

● Nou · resultat · provat

Exploració: cost d'un fil, Peterson en un x86 i el futex

Enunciat. Tres mesures per comprovar (o desmentir) tres afirmacions de les diapositives:

- (1) **create_cost**: crear un fil és més barat que crear un procés? (diapositiva 136).
- (2) **peterson**: l'algorisme de Peterson «és correcte» (diapositiva 152)... en un processador real? Compara-ho amb **peterson_fence**.
- (3) **lock_cost** amb **strace -c**: quantes crides al sistema fa un mutex amb contenció i sense? (diapositives 160–162).

Solució i sortides reals

C tds/td6/create_cost.c

```
/*
 * TD6 - exploration 1: how much does creating a thread cost compared with a
 * process? (original slide 136)
 * Creates and waits for N threads (pthread_create + pthread_join) and N
 * processes (fork + waitpid, the
 * child exits at once) and prints the average time of each.
 * ./create_cost [N]      default: 2000
 */
#include <pthread.h>
```

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void *nothing(void *arg)
{
    return arg;
}

int main(int argc, char *argv[])
{
    int n = argc > 1 ? atoi(argv[1]) : 2000;

    double t0 = now();
    for (int i = 0; i < n; i++) {
        pthread_t t;
        int rc = pthread_create(&t, NULL, nothing, NULL);
        exit_if(rc != 0, "pthread_create");
        exit_if(pthread_join(t, NULL) != 0, "pthread_join");
    }
    double thread_us = (now() - t0) * 1e6 / n;

    t0 = now();
    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            _exit(0);
        exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    }
    double fork_us = (now() - t0) * 1e6 / n;

    printf("%d threads:   %.1f us per pthread_create + pthread_join\n", n,
    thread_us);
  
```

```
printf("%d processes: %.1f us per fork + waitpid\n", n, fork_us);
printf("a process costs %.1f times a thread\n", fork_us / thread_us);
return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

Un fil no copia taules de pàgines ni descriptors: només reserva una pila i una estructura de tasca. A Linux tots dos es creen amb `clone`; el que canvia és què es comparteix.

C tds/td6/peterson.c

```
/*
 * TD6 - exploration 2: Peterson's algorithm on a real multicore x86
 * (original slides 152 and 154).
 * Two threads increment a shared counter N times each, protecting the
 * increment with Peterson's lock.
 * The algorithm is correct on paper, but it "assumes memory accesses are
 * sequentially ordered per thread"
 * (slide 154). Without a fence, the x86 store buffer lets each thread read
 * the other's flag BEFORE its own
 * store is visible: both enter the critical section and increments are lost.
 * With a full fence
 * (mfence, via __atomic_thread_fence) the algorithm works. Compiled twice:
 * peterson (no fence) and
 * peterson_fence (-DFENCE).
 * ./peterson [N]      default: 2000000
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

static volatile int flag[2];
static volatile int turn;
static volatile long counter;
static long n;
```

```

static void *worker(void *arg)
{
    int me = (int)(long)arg, other = 1 - me;
    for (long i = 0; i < n; i++) {
        flag[me] = 1;
        turn = other;
#ifdef FENCE
        __atomic_thread_fence(__ATOMIC_SEQ_CST);    /* the stores must be
visible before the loads */
#endif
        while (flag[other] && turn == other)
            ;    /* busy waiting */
        counter = counter + 1;    /* critical section */
        flag[me] = 0;
    }
    return NULL;
}

int main(int argc, char *argv[])
{
    n = argc > 1 ? atol(argv[1]) : 2000000;
    pthread_t t[2];
    for (long i = 0; i < 2; i++)
        exit_if(pthread_create(&t[i], NULL, worker, (void *)i) != 0,
"pthread_create");
    for (int i = 0; i < 2; i++)
        exit_if(pthread_join(t[i], NULL) != 0, "pthread_join");
#ifdef FENCE
    const char *variant = "with fence";
#else
    const char *variant = "no fence";
#endif
    printf("Peterson %-10s: counter = %ld, expected %ld, lost %ld\n",
variant, counter, 2 * n, 2 * n - counter);
    return EXIT_SUCCESS;
}

```

Sortida real

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do
./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

Sense barrera, algunes execucions perden increments: tots dos fils eren alhora a la secció crítica. L'x86 deixa cada escriptura en una **memòria intermèdia** i permet que un fil llegeixi la nota de l'altre abans que la seva sigui visible (el model TSO).

`__atomic_thread_fence` (una instrucció `m fence`) ho impedeix. És el que adverteix la diapositiva 154 i el que verifica l'animació A7.5, estat per estat.

C tds/td6/lock_cost.c

```
static void *worker(void *arg)
{
    for (long i = 0; i < n; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return arg;
}
```

Sortida real

```
$ ./lock_cost 1; ./lock_cost 8
1 thread(s) x 1000000 lock/unlock: counter = 1000000, 28.2 ns per lock/unlock
8 thread(s) x 1000000 lock/unlock: counter = 8000000, 184.2 ns per
lock/unlock
```

Sortida real

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e
trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00  0.223730    27    8013    3116 futex
```

Sense contenció, 100.000 `lock` / `unlock` no fan **cap** crida a `futex` (l'única és la del `join`): un mutex lliure es pren amb una instrucció atòmica a l'espai d'usuari. Només quan un fil ha d'esperar entra al nucli. És l'«espera activa limitada» de la diapositiva 160, feta realitat pel `futex` de Linux.

Mira-ho verificat → **A7.5 el problema de la llet**

Llista de comprovació

Repte amb IA generativa · Filòsofs: detecta l'interbloqueig amb un graf



Objectiu. Instrumenta `philosophers.c`` per registrar qui espera qui i detectar automàticament el cicle del graf d'espera; compara dues solucions (ordre global i un semàfor «cambrer»).

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Tinc el sopar dels filòsofs amb mutex [enganxa `philosophers.c`]. Ajuda'm a: (1) afegir un fil vigilant que cada 100 ms construeixi el graf «el filòsof i espera el bastonet que té j» i detecti un cicle; (2) implementar la solució del cambrer (un semàfor que deixa seure N-1 filòsofs) i comparar-la amb la de l'ordre global en menjar per segon; (3) relacionar cada solució amb la condició de Coffman que trenca. Abans de programar, pregunta'm quina de les 4 condicions trenca cadascuna.

Com comprovar el resultat

- El fil vigilant detecta el cicle a `philosophers_deadlock`` i imprimeix els 8 filòsofs del cicle.
- Totes dues solucions acaben sempre (repeteix-ho 20 vegades).
- Les teves xifres de menjars per segon surten d'una mesura, no de la IA.

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.